Student Grade Calculator

1. Project Overview

The Student Grade Calculator is a Python console-based application developed as part of Phase 1 – Module 1. The application demonstrates the use of Python fundamentals such as variables, data types, input/output, control structures, loops, functions, error handling, and string formatting to calculate and display student grade reports.

2. Problem Statement

Managing student grades manually can be time-consuming and prone to calculation errors. This project automates the process by collecting student information, validating marks, calculating grades and performance statistics, and generating a structured report using Python.

3. Features

Input Management

- Student details collection
- Subject names entry
- Marks validation

Performance Analysis

- Total marks calculation
- Average marks calculation
- Highest and lowest marks identification
- Grade distribution analysis

Report Generation

- Subject-wise report
- Scholarship eligibility status
- Appreciation remarks
- Formatted student performance report

4. Technologies Used

- Python 3
- Visual Studio Code

5. Program Workflow

- Start
- Display Welcome Message
- Collect Student Details (Name, Student ID, Number of Subjects)
- Validate Inputs
- Collect Subject Names and Marks
- Validate Marks (0–100)
- Calculate Results (Total, Average, Highest, Lowest, Grade, Pass Count)
- Generate Student Performance Report
- Calculate Grades for Another Student?
  - **Yes** → Return to Collect Student Details
  - **No** → Display Thank You Message
- End

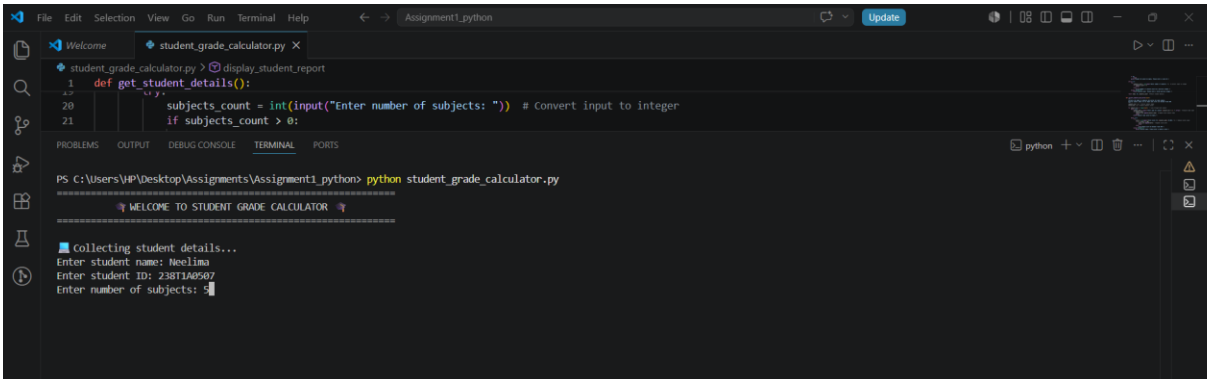
6. Output Screenshots and Explanation

Welcome Screen

Explanation:

The application starts by displaying a welcome message. It introduces the Student Grade Calculator and indicates that the program is ready to collect student information.

Student Details Input



Explanation:

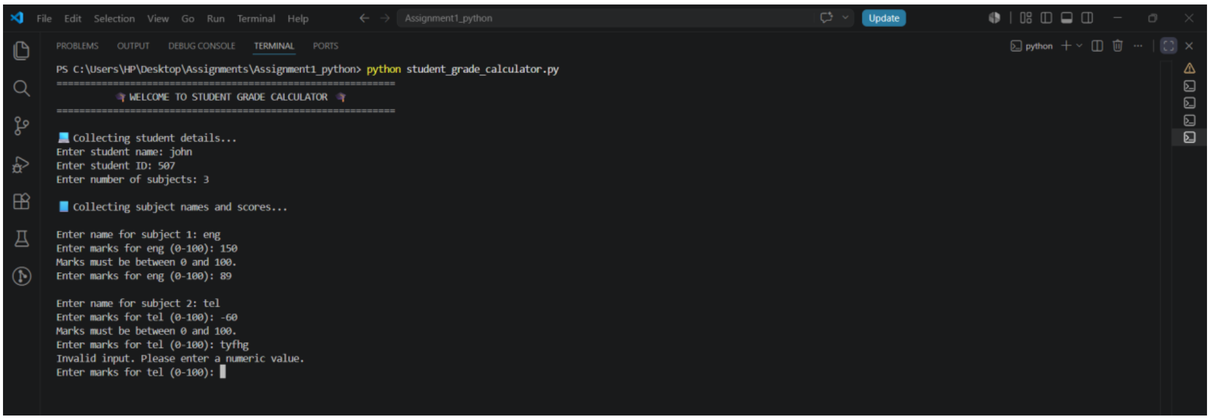
The application collects the student's basic information. Input validation ensures that the student's name and ID are not empty and that the number of subjects is a positive integer.

Subject Marks Entry

Explanation:

The user enters each subject name and its corresponding marks. The application validates that marks are within the range of 0–100 and accepts only valid numeric input.

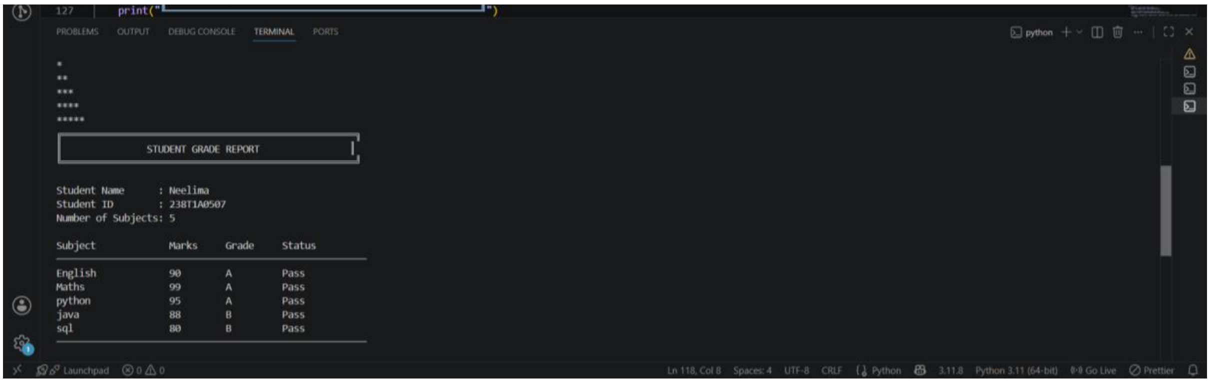
Input Validation



Explanation:

The application uses exception handling and input validation to detect invalid entries. Appropriate error messages are displayed, prompting the user to provide valid input without terminating the program.

Generated Report



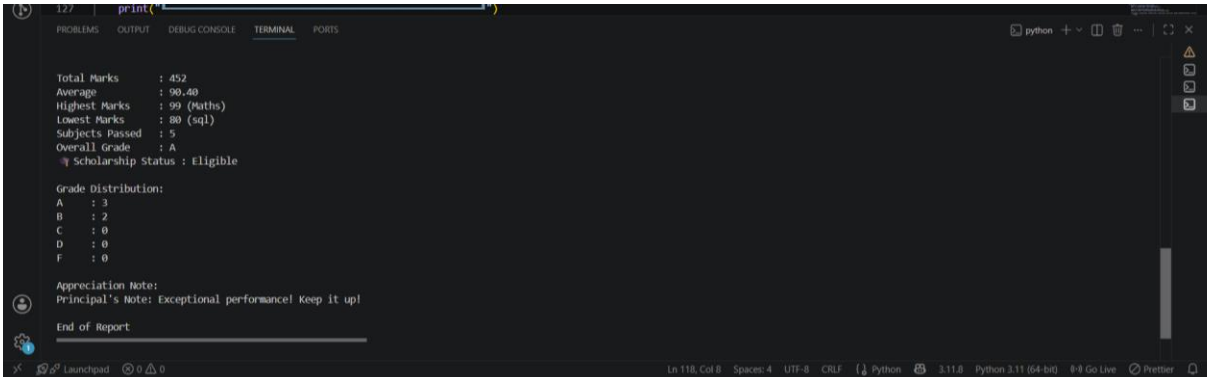
Explanation:

The generated report displays the student's information, subject-wise marks, grades, pass/fail status, and other academic details in a well-formatted table using formatted strings.

- Student details
- Subject-wise marks
- Grade for each subject
- Pass/Fail status

The information is displayed in a neatly formatted table using formatted strings.

Statistical Summary



Explanation:

This section displays the student's overall academic performance by summarizing the calculated results and additional insights. It provides a clear evaluation of the student's achievements and highlights their overall progress.

- Total Marks
- Average Marks
- Highest Marks
- Lowest Marks
- Subjects Passed
- Overall Grade
- Scholarship Status
- Grade Distribution
- Appreciation Note

Multiple Student Option

Explanation:
The application uses a while loop to allow users to calculate grades for multiple students in a

single execution. When the user selects "no", the application terminates gracefully with a thank-you message.

7. Learning Outcomes

- Practiced modular programming
- Applied exception handling
- Improved input validation techniques
- Used functions for better code organization
- Strengthened problem-solving skills
- Learned to generate formatted console reports

8. Conclusion

This project strengthened my understanding of Python fundamentals by combining concepts such as functions, loops, conditional statements, exception handling, lists, and formatted output into a single real-world application. It also improved my ability to write modular, maintainable, and user-friendly programs.